Assignment 4 Part 2: Practical Exploration of ILP Techniques in gem5

Todd Simonson

University of the Cumberlands

MSCS 531: Computer Architecture and Design

Dr. Gary Perry

March 29, 2026

Assignment 4 Part 2: Practical Exploration of ILP Techniques in gem5

1. Environment Setup and Repository

This assignment builds on the gem5 environment established in Assignment 3. gem5 version 25.1.0.0, compiled March 8, 2026, was used for all simulations. A new private GitHub repository was created at https://github.com/tgsimonson/mscs531-gem5-ilp-analysis to store all configuration files, benchmark source code, and simulation output files.

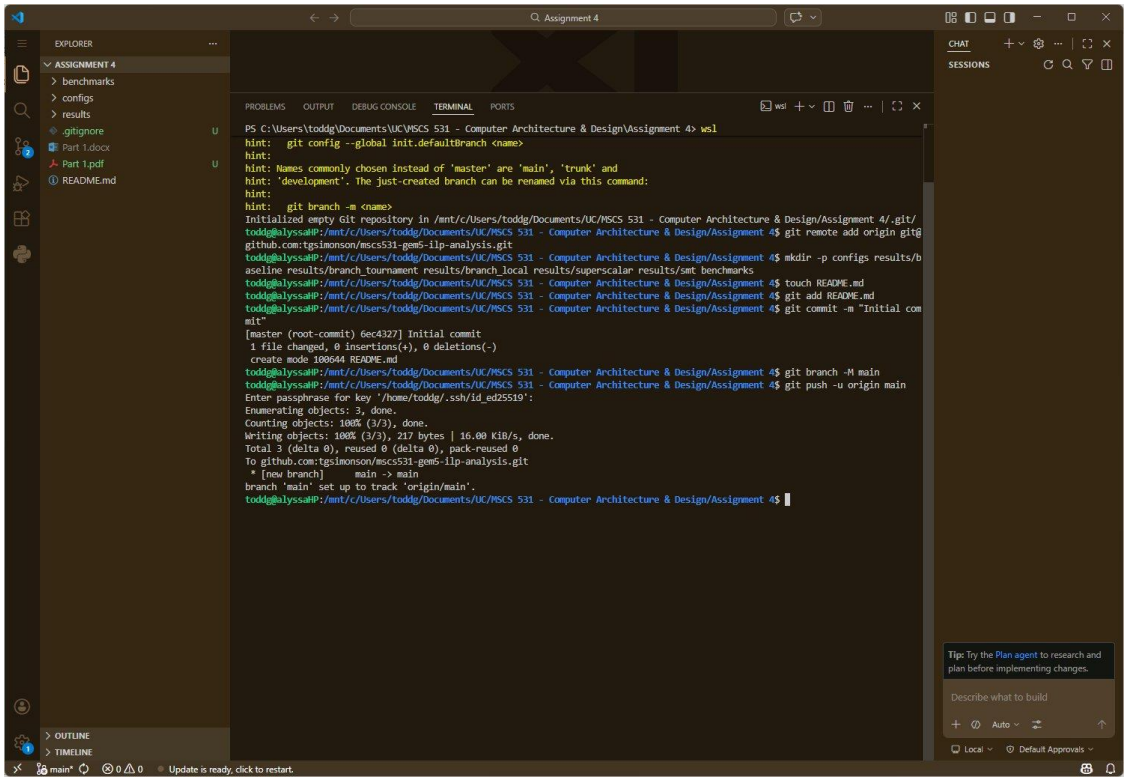

Figure 1. Initial repository push confirming the mscs531-gem5-ilp-analysis repository was created and connected to GitHub.

2. Benchmark Program

The same 64x64 integer matrix multiplication program used in Assignment 3 was selected as the benchmark. Matrix multiplication was chosen because its nested loop structure generates thousands of memory references and branch instructions, producing cache and branch behavior that varies measurably across ILP configurations. Compiler optimizations were disabled with the -O0 flag to preserve natural memory access and control flow patterns. The program was compiled with GCC and verified to produce the correct output natively before being used in gem5 simulations.

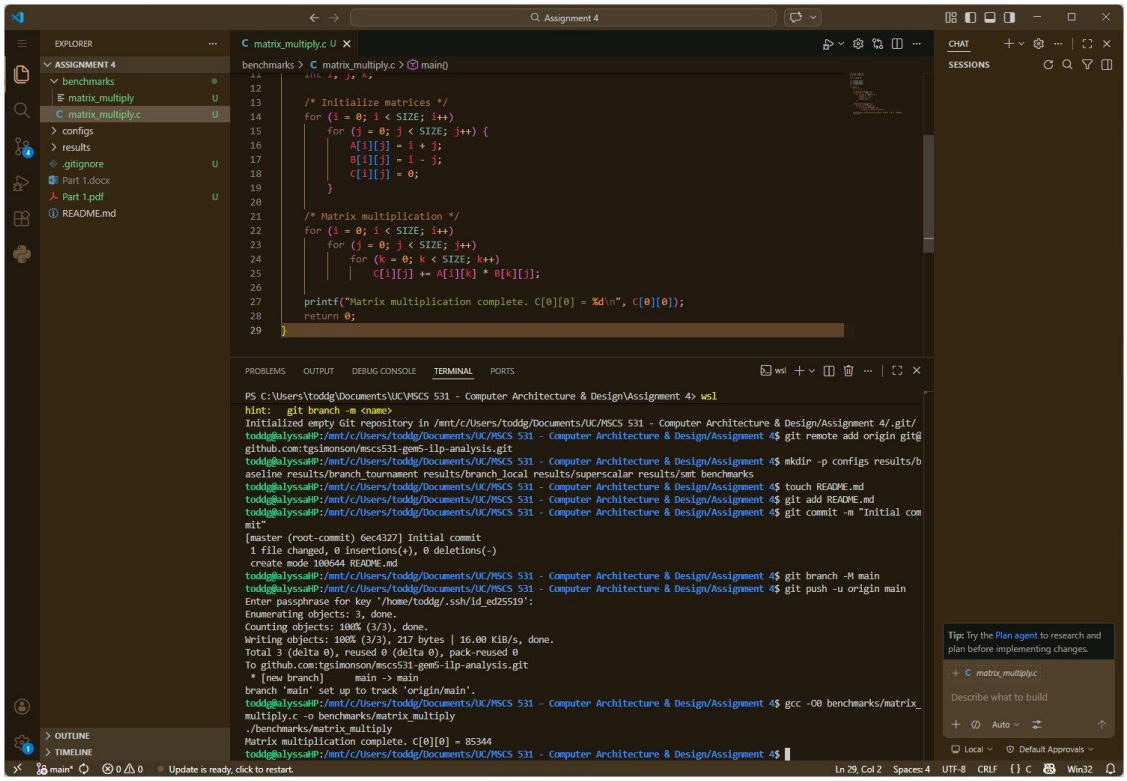

Figure 2. matrix_multiply.c open in VS Code (top panel) with gcc compile command and successful native execution confirming correct output C[0][0] = 85344.

3. Baseline Pipeline Simulation

The baseline configuration uses a TimingSimpleCPU model, which is an in-order single-issue processor that stalls completely on every memory access and has no branch prediction. This represents the performance floor against which all ILP optimizations are compared. The cache configuration matches Assignment 3: 32kB 4-way L1 instruction and data caches with 64-byte blocks, connected to a DDR3_1600_8x8 memory controller with a 512MB address range at 1GHz.

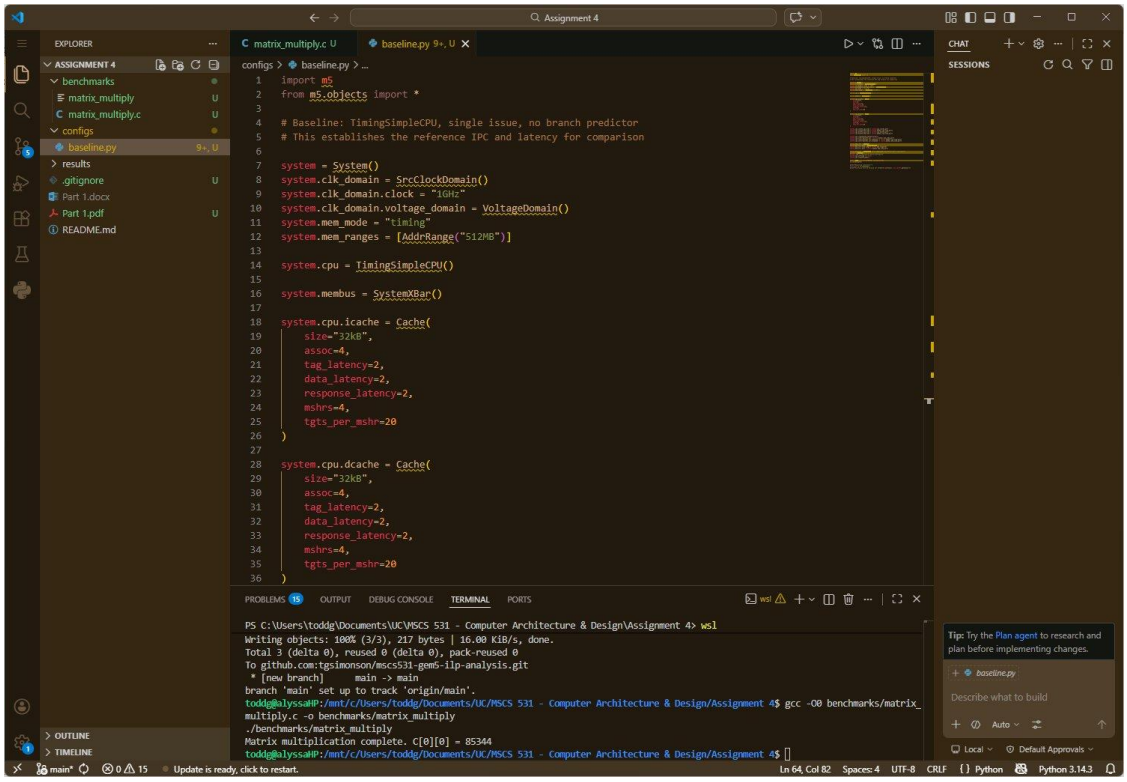

Figure 3. baseline.py configuration file open in VS Code showing the TimingSimpleCPU setup with cache and memory controller parameters.

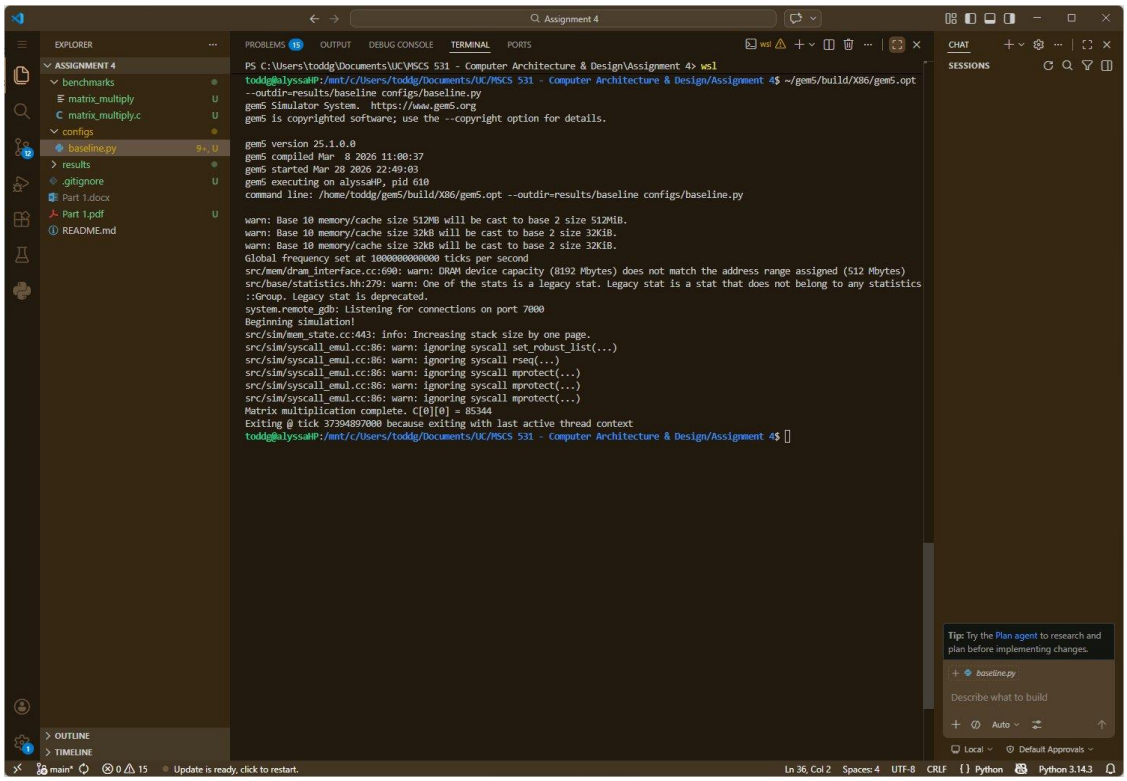

Figure 4. Baseline simulation completing at tick 37,394,897,000. The TimingSimpleCPU stalls on every memory access, producing a high cycle count relative to the O3CPU experiments.

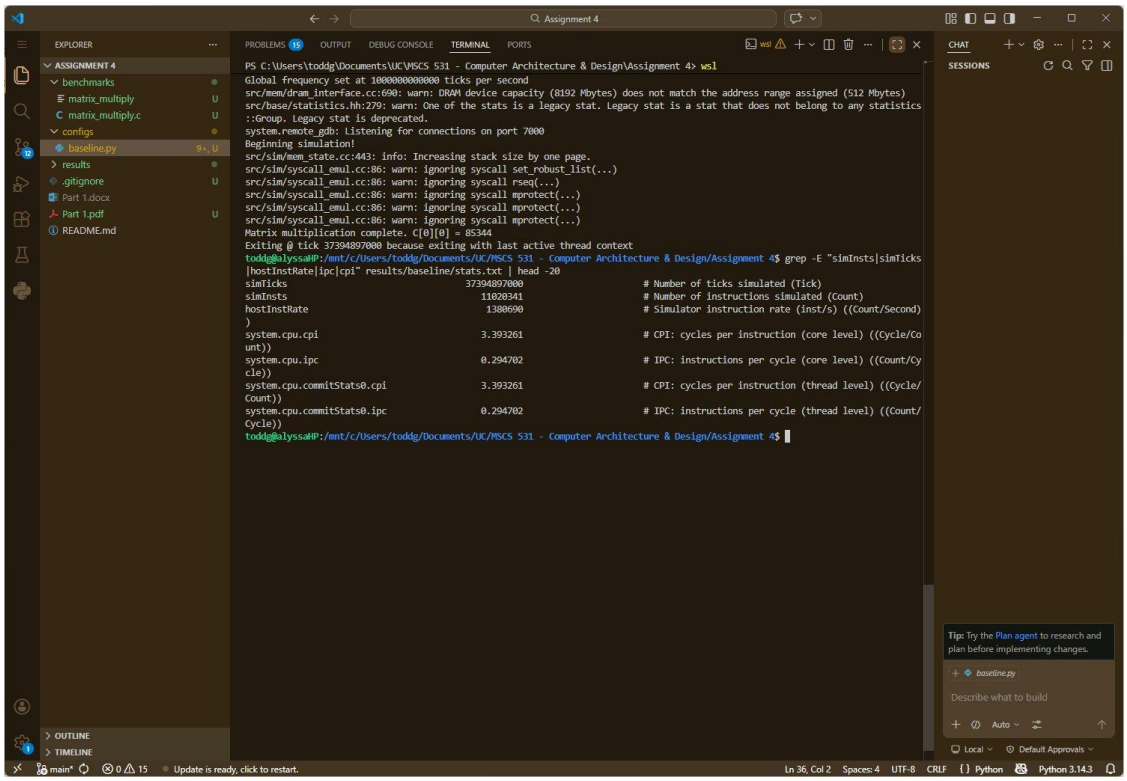

Figure 5. Baseline performance metrics. IPC of 0.2947 reflects the single-issue in-order execution with no branch prediction or out-of-order capability.

The baseline IPC of 0.2947 and CPI of 3.393 establish the reference point. These numbers reflect the fundamental cost of in-order execution: every cache miss stalls the entire pipeline, and every branch instruction must fully resolve before the next fetch begins. This configuration represents how processors behaved before the introduction of out-of-order execution and speculative techniques (Hennessy & Patterson, 2017).

4. Branch Prediction Experiments

4.1 Troubleshooting

Transitioning from TimingSimpleCPU to DerivO3CPU with branch prediction introduced several configuration errors that required iterative debugging. The first error occurred when attempting to instantiate TournamentBP with keyword arguments directly in the constructor, which gem5 25.1 does not support. A second error arose because the branchPred parameter on the O3CPU expects a BranchPredictor wrapper object containing a conditionalBranchPred field, not a direct TournamentBP assignment. These errors were resolved by using the correct nested object structure.

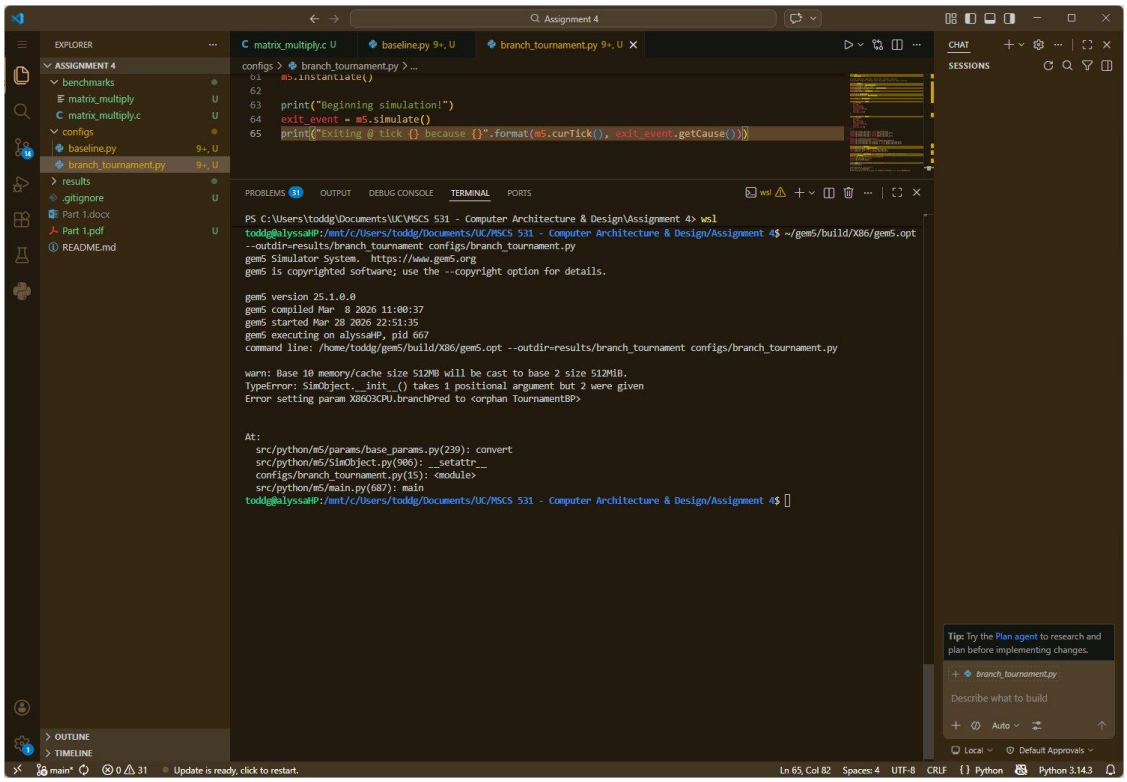

Figure 6. First error: gem5 rejected TournamentBP instantiation with keyword arguments, reporting SimObject.__init__() takes 1 positional argument.

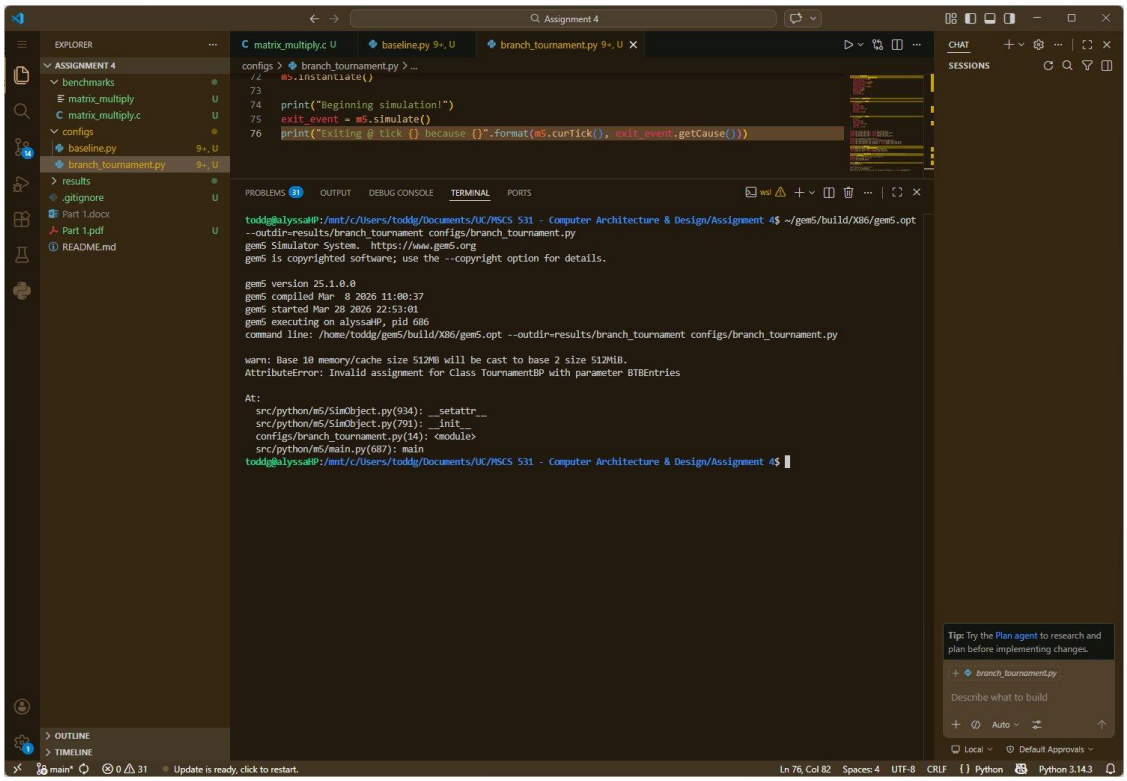

Figure 7. Second error: invalid BTBEntries parameter on TournamentBP. Investigation of gem5 source confirmed TournamentBP only accepts six valid parameters: localPredictorSize, localCtrBits, localHistoryTableSize, globalPredictorSize, globalCtrBits, and choicePredictorSize.

4.2 Tournament Branch Predictor

The tournament predictor combines local and global branch history to select the more accurate prediction for each branch. Local history tracks the pattern of each individual branch instruction, while global history tracks the pattern of recent branches across the entire instruction stream. A choice predictor selects between them based on which has been more accurate for a given branch. The configuration uses the default sizes: 2048-entry local predictor, 8192-entry global predictor, and 8192-entry choice predictor.

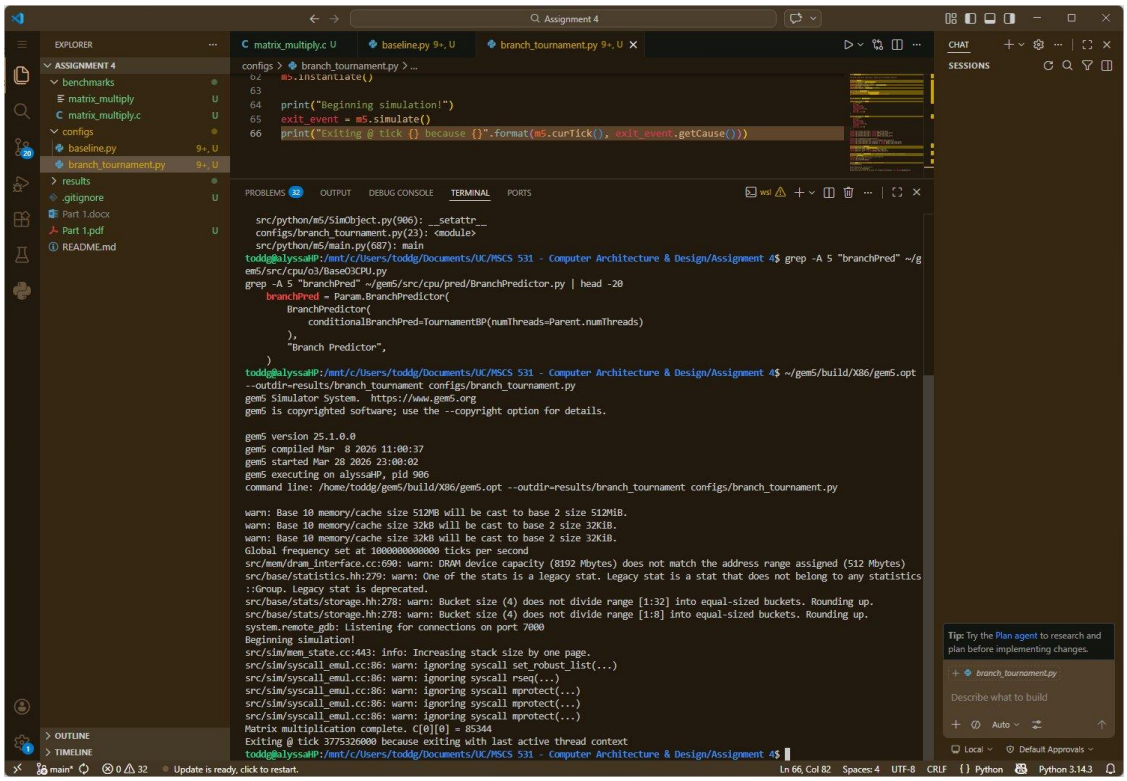

Figure 8. Tournament branch predictor simulation completing at tick 3,775,326,000, nearly 10x faster than the baseline. The shift to DerivO3CPU with out-of-order execution accounts for the majority of this improvement.

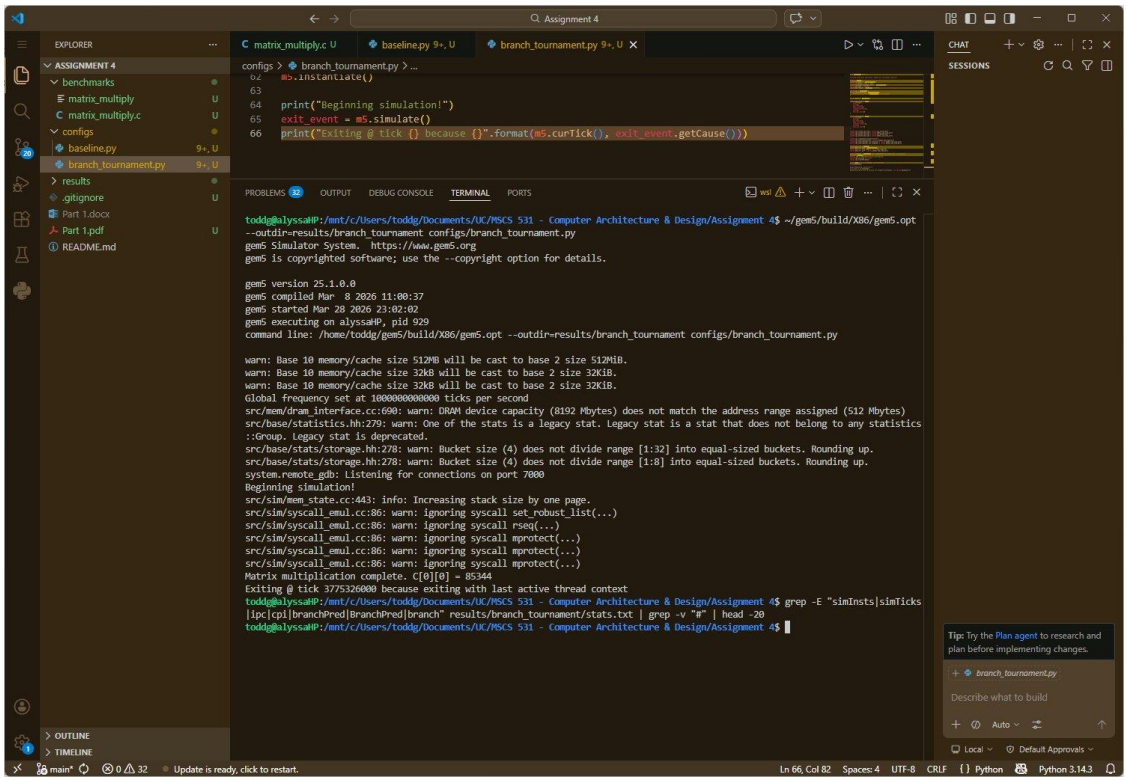

Figure 9. Tournament branch predictor performance statistics. IPC of 2.919 and 7,516 branch mispredictions out of 316,389 total branch lookups, a misprediction rate of approximately 2.4%.

4.3 Local Branch Predictor

The local predictor uses only per-branch history, tracking the last several outcomes for each individual branch instruction without considering global context. This is simpler than the tournament predictor and requires less hardware, but may be less accurate on branches whose behavior correlates with other branches in the instruction stream.

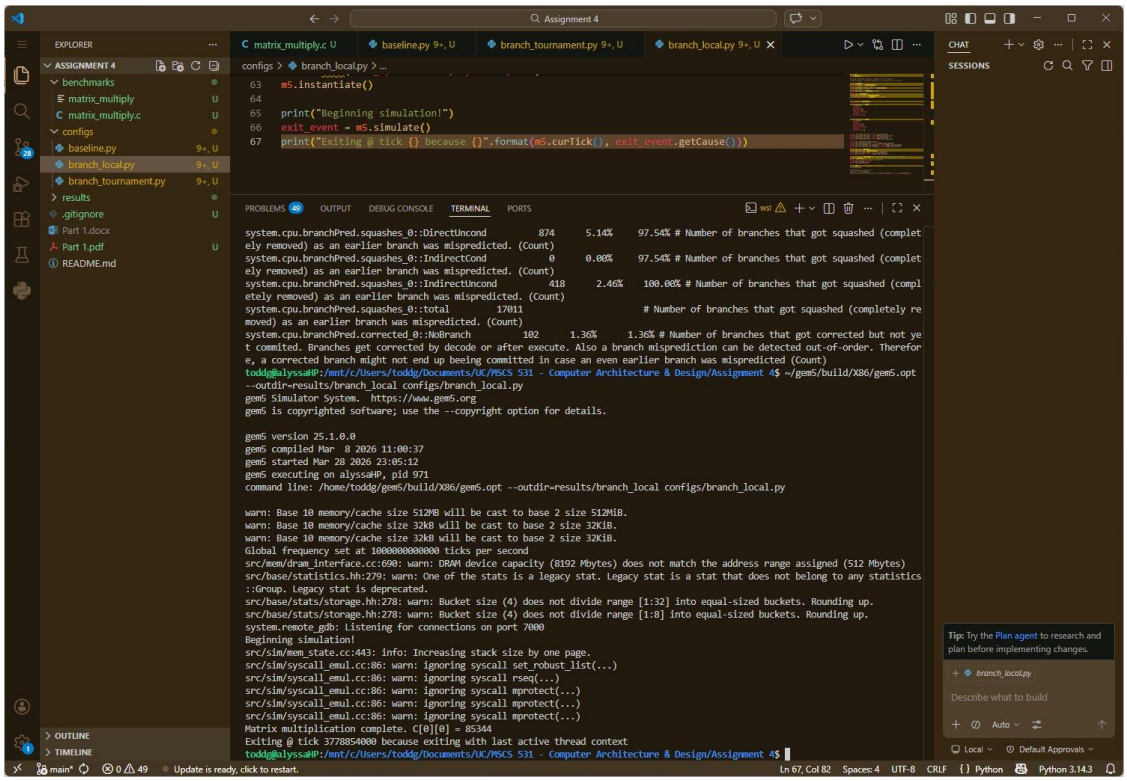

Figure 10. Local branch predictor simulation completing at tick 3,778,854,000, slightly slower than the tournament predictor, consistent with the higher misprediction count.

The local predictor produced 7,932 branch mispredictions compared to 7,516 for the tournament predictor. The IPC of 2.916 is marginally lower than the tournament predictor's 2.919. While the difference is small for this workload, it illustrates the theoretical advantage of combining local and global history: the tournament predictor can select the better mechanism per branch rather than committing to one approach for all branches (Schall et al., 2024).

5. Superscalar Multiple Issue Simulation

The superscalar configuration modifies the O3CPU to issue up to four instructions per cycle by widening the fetch, decode, rename, dispatch, issue, and commit widths to four. All other

parameters remain identical to the tournament branch predictor experiment, including the tournament predictor and cache configuration. This isolates the effect of wider issue width on performance.

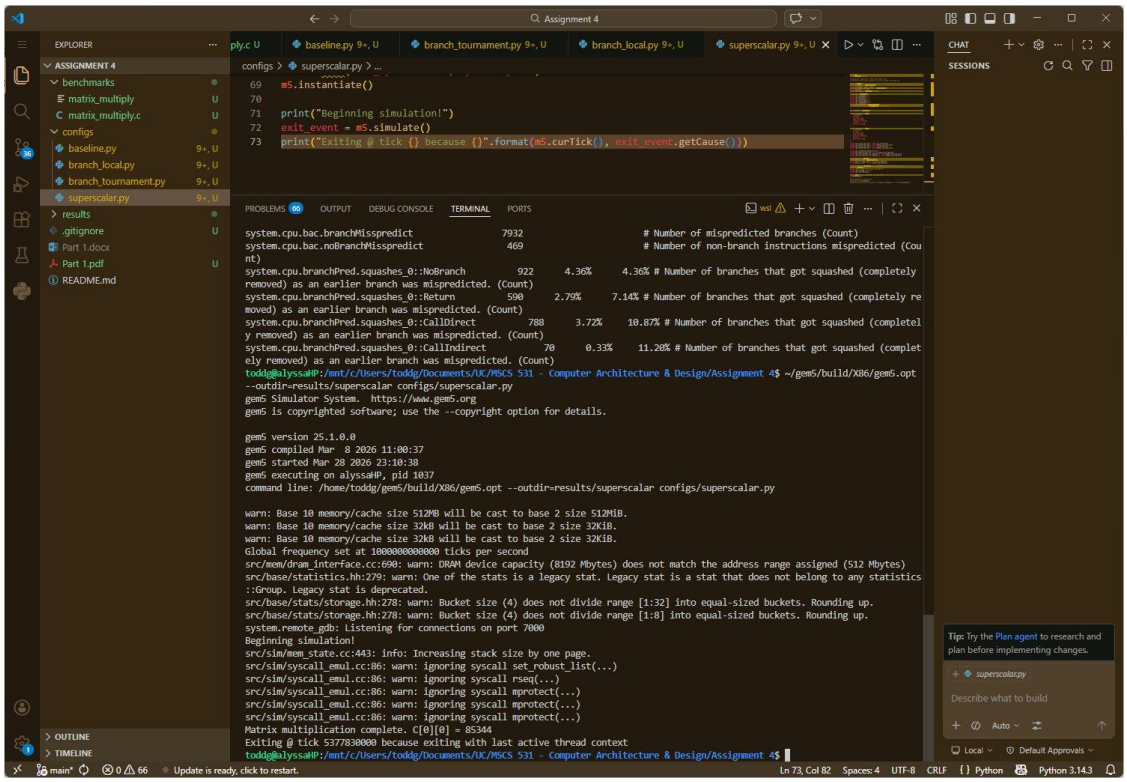

Figure 11. Superscalar simulation completing at tick 5,377,830,000. The higher tick count compared to the single-issue O3CPU experiments is a notable result.

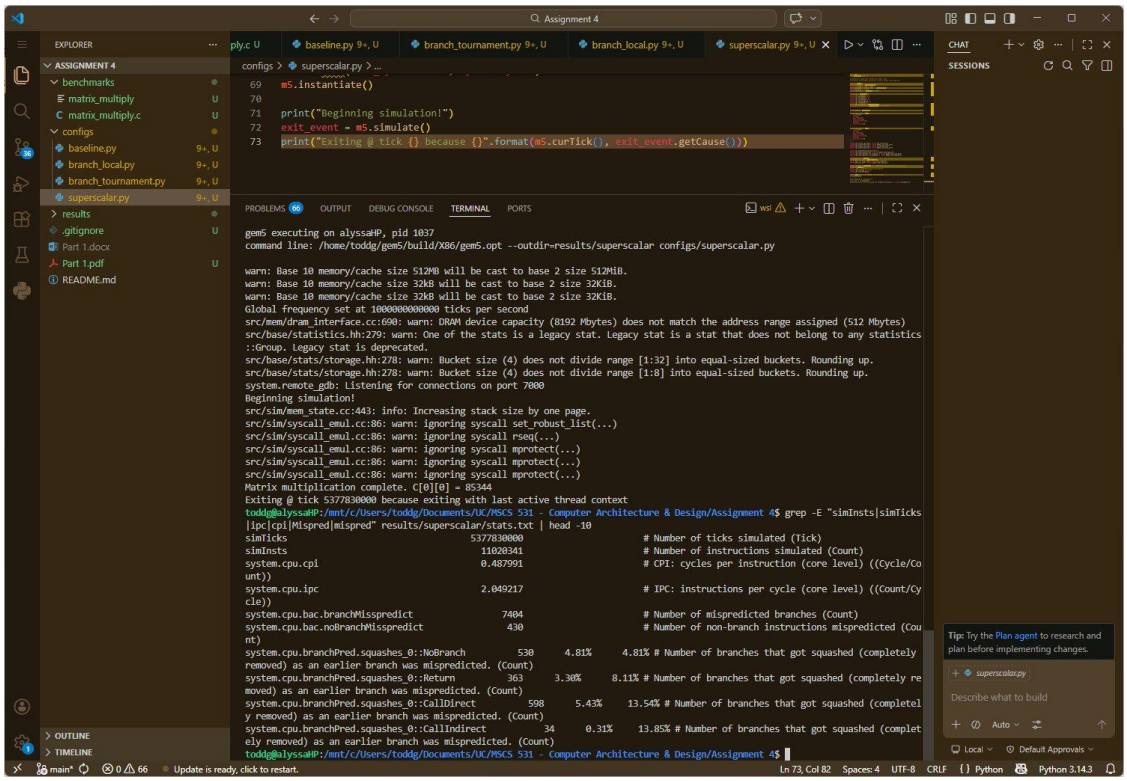

Figure 12. Superscalar performance statistics. IPC of 2.049 is lower than the single-issue O3CPU despite the wider datapath, with 7,404 branch mispredictions.

The superscalar result is the most analytically interesting of all experiments. The 4-wide configuration achieved an IPC of 2.049, which is lower than the single-issue O3CPU's IPC of 2.919. This counterintuitive result directly illustrates the ILP wall: the matrix multiplication workload does not contain enough independent instructions to keep four issue slots simultaneously occupied. The overhead of managing a wider pipeline — including the additional complexity of dependency checking, register renaming across more in-flight instructions, and wider fetch bandwidth — costs more cycles than the occasional benefit of issuing multiple instructions simultaneously. As Hennessy and Patterson (2017) describe, practical ILP rarely exceeds two to three instructions per cycle even with wide superscalar datapaths, because true data dependencies in real programs create unavoidable serialization.

6. Simultaneous Multithreading

6.1 Troubleshooting

Configuring SMT in gem5 25.1 required resolving three separate errors. The first was a PID conflict when both processes defaulted to the same process ID; this was fixed by explicitly assigning pid=100 and pid=101 to each process. The second was an assertion failure because the system did not have multithread mode enabled; this required setting system.multi_thread = True before CPU instantiation. The third was an interrupt controller connection error because the default interrupt setup only connected one thread's interrupt ports; this required a loop connecting ports for both threads.

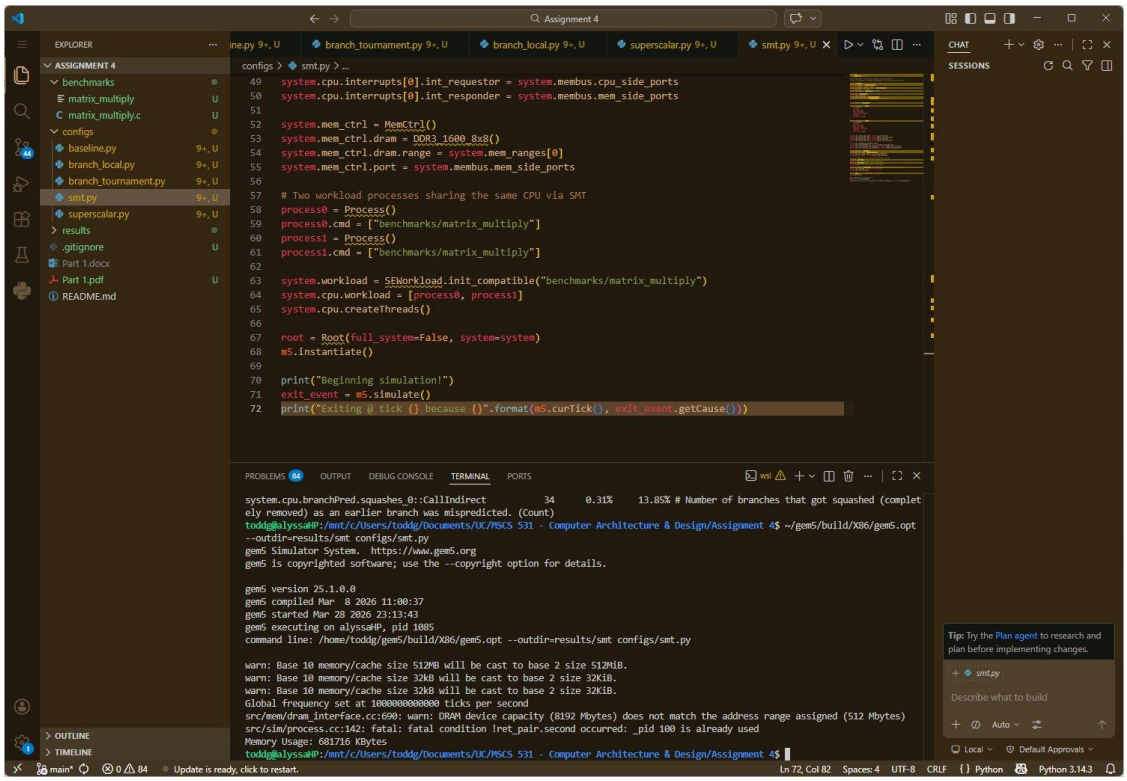

Figure 13. SMT configuration errors encountered during setup, including PID conflict and multiThread assertion failure. Each was diagnosed from the error message and resolved iteratively.

6.2 SMT Results

The final SMT configuration runs two hardware threads on a single O3CPU, each executing an independent matrix multiplication workload. Both threads share the reorder buffer, execution units, caches, and branch predictor resources.

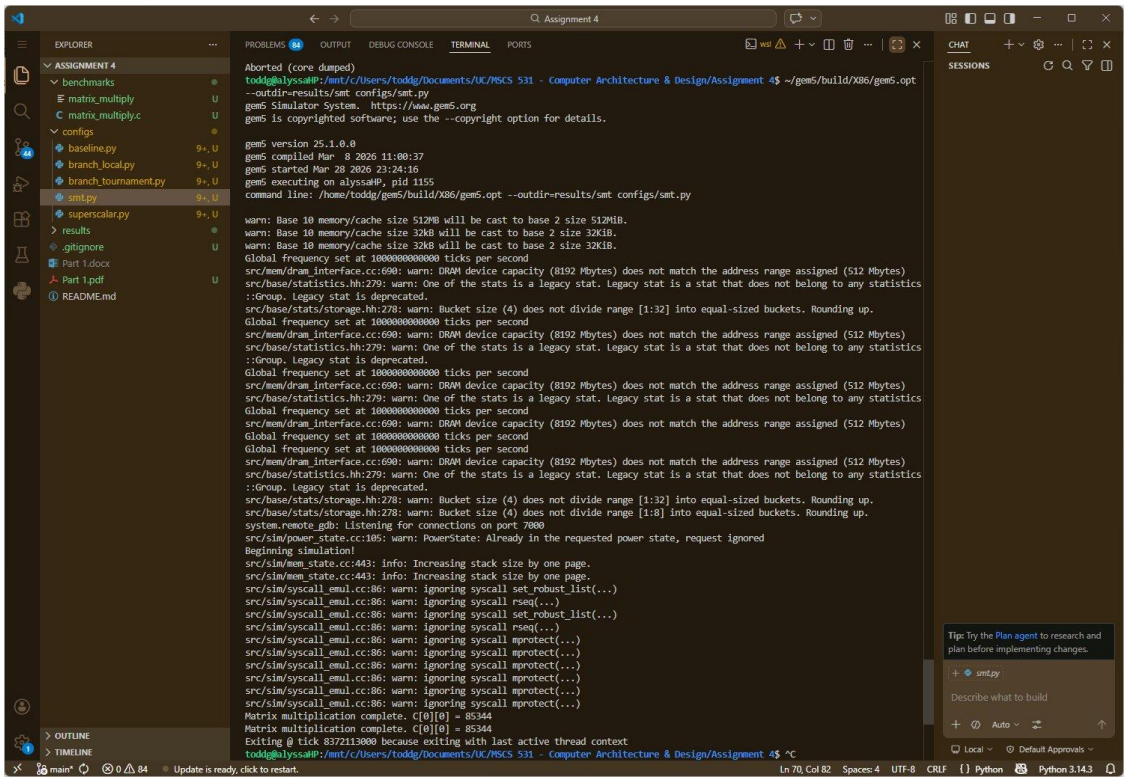

Figure 14. SMT simulation completing at tick 8,372,113,000 with both threads printing their matrix multiplication results, confirming both workloads executed correctly.

The SMT experiment executed 22,040,682 total instructions — exactly twice the single-thread count — in 8,372,113,000 ticks. The aggregate core IPC was 2.633, while each individual thread achieved an IPC of 1.316. Two threads completed in approximately 2.2x the time of one thread alone, meaning aggregate throughput increased by roughly 1.8x compared to running a single thread. This is consistent with what Hennessy and Patterson (2017) describe as the primary benefit of SMT: converting latency that would otherwise stall the pipeline into productive work by switching to a ready thread. The per-thread IPC reduction from 2.919 to 1.316 reflects resource contention between the two threads competing for the same reorder buffer entries, cache bandwidth, and execution units.

7. Results Summary and Analysis

Table 1. ILP Experiment Performance Comparison

Metric	Baseline	Tournament BP	Local BP	Superscalar	SMT (2T)
Sim ticks	37,394,897,000	3,775,326,000	3,778,854,000	5,377,830,000	8,372,113,000
Instructions	11,020,341	11,020,341	11,020,341	11,020,341	22,040,682
IPC (core)	0.2947	2.9190	2.9163	2.0492	2.6326
CPI (core)	3.3933	0.3426	0.3429	0.4880	0.3798
Branch mispredictions	N/A	7,516	7,932	7,404	N/A

Table 1. Performance metrics across all five gem5 simulation configurations.

The results reveal a clear hierarchy of ILP effectiveness for this workload. The transition from in-order TimingSimpleCPU to out-of-order O3CPU with tournament branch prediction produced the largest single improvement, a 9.9x IPC gain, driven by out-of-order execution's ability to find independent instructions and hide memory latency. Branch predictor type had minimal impact on this workload, with tournament and local predictors performing nearly identically. Wider superscalar issue actually reduced IPC due to the workload's limited instruction-level parallelism. SMT delivered meaningful aggregate throughput gains at the cost of per-thread performance.

These findings directly connect to the theoretical concepts from Part 1. The ILP wall, described by Hennessy and Patterson (2017) as the fundamental limit on extractable parallelism, is visible in the superscalar result. Amdahl's Law applies: the serial portions of matrix multiplication that involve true data dependencies cannot be parallelized regardless of issue width. The branch prediction comparison illustrates what Schall et al. (2024) document in server

workloads — predictor accuracy matters most for hard-to-predict branches, and for a regular workload like matrix multiplication, the difference between predictor types is small. The SMT experiment confirms that thread-level parallelism can recover the throughput that ILP cannot provide, converting pipeline latency into useful work from a second thread.

8. GitHub Repository

All configuration files, the benchmark source code, and simulation output files are stored in the private GitHub repository at https://github.com/tgsimonson/mscs531-gem5-ilp-analysis. The repository includes a README documenting the experiment setup and how to reproduce each simulation.

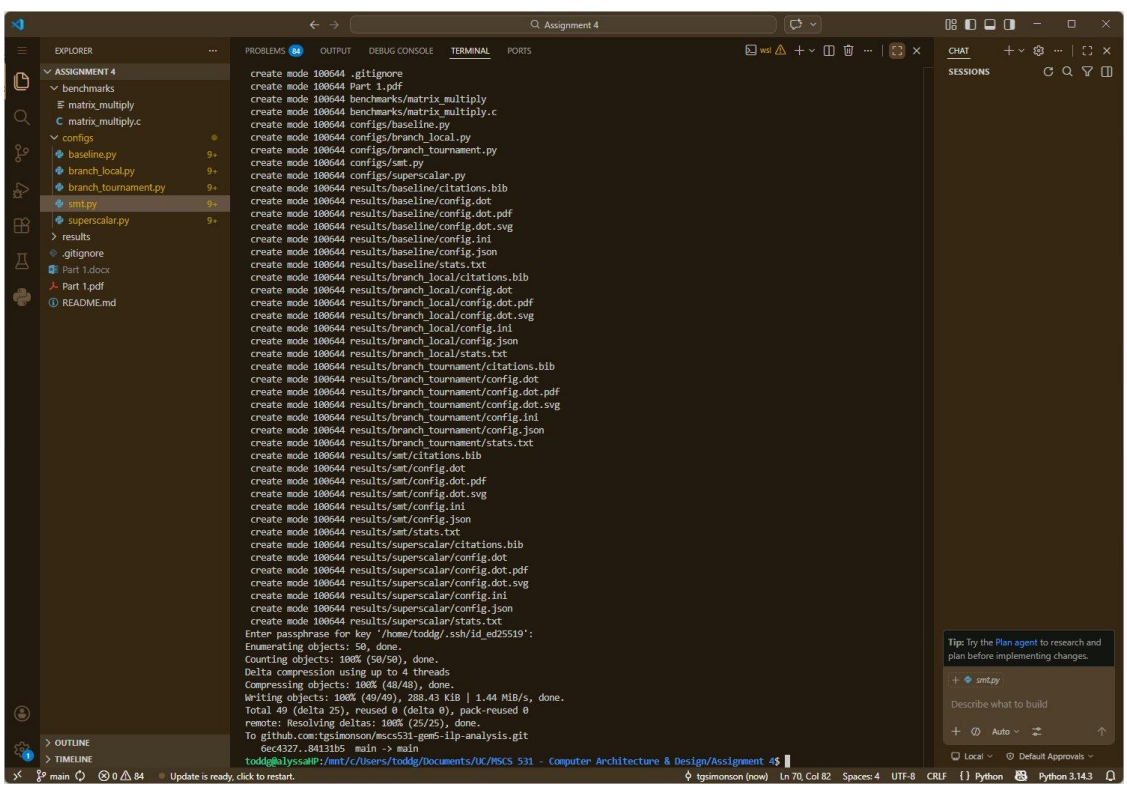

Figure 15. Git push confirming all configuration files and simulation results were successfully committed and pushed to the private GitHub repository.

9. Troubleshooting Summary

Three categories of configuration errors were encountered across the experiments. First, branch predictor instantiation in gem5 25.1 requires a BranchPredictor wrapper object with a conditionalBranchPred field rather than direct assignment of a predictor type to branchPred. Second, SMT configuration requires system.multi_thread = True set at the system level before CPU instantiation, explicit PID assignment for each process, and interrupt controller port connections looped across all threads. Third, the superscalar configuration revealed that wider issue width does not guarantee higher IPC — workload characteristics determine whether additional issue slots can be utilized. Each error was documented, diagnosed from the error message, and resolved before proceeding, demonstrating the iterative debugging process that is central to architectural simulation work.

References

Hennessy, J. L., & Patterson, D. A. (2017). *Computer architecture* (6th ed.). Elsevier S & T. https://reader.yuzu.com/books/9780128119068

Schall, D., Sandberg, A., & Grot, B. (2024). The last-level branch predictor. *Proceedings of the 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. https://doi.org/10.1109/MICRO61859.2024.00042

gem5 Project. (2025). *gem5 simulator documentation*. https://www.gem5.org/documentation/